

Online Resource 1

Supplement to “Pre-Export Row Scaling for Generated Mixed-Integer Programs: Mechanism Attribution and Residual-Budget Contracts”

Mathias Rodríguez Castro

Facultad de Ingeniería, Universidad de la República, Montevideo, Uruguay

mathiasr@fing.edu.uy

Prepared for Mathematical Programming Computation

This supplement specifies the computational evidence and its limitations. Paths are relative to the accompanying repository. Frozen results can be checked without an optimizer license. The earlier draft and intermediate runs are retained to make corrections traceable; they are not interchangeable with the final datasets.

S1 Proof for the triangular model

For $R \in \mathbb{R}^{m \times n}$, $m, n \geq 1$, set $\rho = \|R\|_2$ and

$$C_\gamma = \begin{bmatrix} I_n & 0 \\ \gamma R & \gamma I_m \end{bmatrix}, \quad \gamma > 0.$$

The main-text proposition states that the unique optimal common coupling factor is $(1 + \rho^2)^{-1/2}$ and the minimum condition number is $\sqrt{1 + \rho^2} + \rho$. The interval-constrained optimum clips this factor to the interval.

Proof. An SVD of R and orthogonal transformations reduce the nontrivial spectrum to blocks $\begin{bmatrix} 1 & 0 \\ \gamma\sigma & \gamma \end{bmatrix}$, one for each singular value σ of R , with possible additional singular values 1 or γ . Each block’s squared singular values have product γ^2 and sum $1 + \gamma^2(1 + \sigma^2)$. Their largest value increases and smallest value decreases with σ ; the block at $\sigma = \rho$ therefore determines both extrema, including any additional values. If k is their singular-value ratio, then

$$k + k^{-1} = \frac{1 + \gamma^2(1 + \rho^2)}{\gamma} = \gamma^{-1} + \gamma(1 + \rho^2).$$

The left-hand side is increasing for $k \geq 1$. The right-hand side has a unique minimum at $\gamma_* = (1 + \rho^2)^{-1/2}$, decreases before that point, and increases after it. At the minimum, $k + k^{-1} = 2\sqrt{1 + \rho^2}$, whose root $k \geq 1$ is $\sqrt{1 + \rho^2} + \rho$. The interval result follows from the same monotonicity. For $\rho = 0$, the argument reduces to balancing the diagonal entries at one. $\square$

This scalar result assumes already whitened local blocks and one common coupling factor. It does not solve general diagonal preconditioning and says nothing directly about simplex bases or mixed-integer search.

S2 Fixed-basis conditioning audit

Protocol. The protocol was fixed after a pilot on the first five inputs of each class (`fixed-basis-recovery-20260914-v2/`) and then applied, unchanged except for the policy set of the manuscript, to all 306 archived dispatch inputs with no exclusion: 204 Simple, 68 SG-Ter-Mer and 34 Full models. The audit build of the C++ generator writes five LP files per model through its solver-free exporter, with the following preprocessing flags.

Policy	Flags of <code>preprocesar</code>
Base	<code>--solodiagnostico</code>
Flat-GM	<code>--local-matrical --plano</code>
Flat-L2	<code>--local-matrical --plano --kernel-euclidean</code>
Role-Hybrid	<code>--local-matrical --acoplamiento-l2</code>
SA-Mat	<code>--local-matrical</code>

HiGHS 1.15.1 reads Base with small-matrix threshold 10^{-12} , drops integrality and solves the LP relaxation with the simplex method (one thread, random seed 0, 300-second limit); all 306 solves are optimal. The basis is stored by row and column names.

Each variant is read with the same reader and compared with Base before assembly: identical named rows and columns, sparsity pattern, bounds, objective, offset and integrality. A row’s factor is inferred from its largest-magnitude coefficient, and the whole row and both RHS values must agree with that factor to relative tolerance 2×10^{-13} ; the largest observed deviation is 2.5×10^{-16} . The basis consists of the basic structural columns of the variant’s own matrix and a unit column for every basic slack.

Estimate and verification. $\|B\|_1$ is computed from the stored entries, and $\|B^{-1}\|_1$ is estimated by SciPy’s `onenormest` ($t = 4$) on solves with a SuperLU factorization, with seeds 0, 1 and 2; the tables use seed 0. The seeds agree within a factor of 2.4, and 63 of 1,530 records differ by more than 1%. The largest componentwise backward error of a diagnostic solve is 1.9×10^{-8} . The script `scripts/research/verify_fixed_basis.py` imports neither the driver nor the HiGHS reader. It checks the recorded hashes, parses every LP file as text, rebuilds each basis from the stored names and compares it entry by entry with the stored matrix: all 1,530 bases match exactly. It then recomputes each estimate as $B^{-1} = Q^{-1}D$, with $Q = DB$ and D an exact power-of-two row equilibration. The two routes differ by less than 10^{-6} in 1,512 records and by at most 23% in all; since their test solutions agree to 10^{-12} in every record, the remaining differences point to the randomized estimator’s iteration path rather than to the solves.

Interpretation. Row scaling changes κ_1 , so the audit measures the exported representation before any solver-internal scaling. The Base estimates exceed the reciprocal of machine precision; the precision check below shows that they are nevertheless accurate on a stress sample. Every row of the 306 models has a coefficient of magnitude at least one, and $1/\|a_r\|_2 \leq 1/\max_j |a_{rj}|$, so no Flat-L2 factor exceeds one. Across all rows, Flat-GM enlarges 22.5% (Simple), 32.1% (SG-Ter-Mer) and 27.3% (Full) and leaves 30.0%, 22.0% and 24.5% unscaled; Flat-L2 enlarges none and leaves 57.6%, 64.3% and 59.2% unscaled. SA-Mat and Role-Hybrid exports are byte-identical on all 306 models. SA-Mat coincides with Flat-GM on every Simple model and, in each SG-Ter-Mer and Full model, differs from it only on the 1,440 `falla_tramo` market-shortage rows, which SA-Mat and Role-Hybrid leave unscaled. The authoritative directory is `results-revision/research-audit/fixed-basis-final-20260914/`, with design, source snapshot, exports, bases, per-row factors, verification and `analysis.json`.

Table S1: Fixed-basis audit: per-instance ratios $\kappa_1(\text{policy})/\kappa_1(\text{Base})$ on the common basis. “Improved” counts ratios below one.

Class	Policy	n	Median ratio	Minimum	Maximum	Improved
Simple	Flat-GM	204	2.4×10^{-7}	1.9×10^{-8}	1.7×10^{-3}	204
Simple	Flat-L2	204	1.57	3.3×10^{-4}	2.7×10^2	32
Simple	Role-Hybrid	204	2.4×10^{-7}	1.9×10^{-8}	1.7×10^{-3}	204
Simple	SA-Mat	204	2.4×10^{-7}	1.9×10^{-8}	1.7×10^{-3}	204
SG-Ter-Mer	Flat-GM	68	7.3×10^{-9}	1.5×10^{-9}	1.2×10^{-2}	68
SG-Ter-Mer	Flat-L2	68	1.51	8.1×10^{-4}	18.61	10
SG-Ter-Mer	Role-Hybrid	68	7.3×10^{-9}	1.5×10^{-9}	0.57	68
SG-Ter-Mer	SA-Mat	68	7.3×10^{-9}	1.5×10^{-9}	0.57	68
Full	Flat-GM	34	3.5×10^{-7}	9.3×10^{-9}	3.1×10^{-6}	34
Full	Flat-L2	34	1.49	0.85	1.5×10^2	2
Full	Role-Hybrid	34	3.5×10^{-7}	9.3×10^{-9}	3.1×10^{-6}	34
Full	SA-Mat	34	3.5×10^{-7}	9.3×10^{-9}	3.1×10^{-6}	34

Precision check. Because the unscaled values exceed the reciprocal of machine precision, the script `scripts/research/verify_fixed_basis_precision.py` checks their accuracy on a stress sample fixed before computation: in each class, the model with the largest Base estimate, the model with the largest Flat-GM/Base estimated ratio, and one other model drawn with seed 20260915. In Full the first two coincide (instance014), which gives eight models and 24 bases under Base, Flat-GM and Flat-L2. For each hash-checked basis, $\|B\|_1$ is computed in exact rational arithmetic. Every column of B^{-1} is solved in double precision; with x_j the computed column and ρ_j a bound on $\|Bx_j - e_j\|_1$ that includes the rounding-error terms of the residual evaluation, $\|B^{-1}\|_1 \leq \max_j \|x_j\|_1 / (1 - \max_j \rho_j)$ whenever $\max_j \rho_j < 1$. This upper bound is evaluated in floating point with explicit error terms rather than directed rounding. For the five largest columns, the solution is refined with residuals computed exactly in rational arithmetic on the stored binary64 data; since $\|B^{-1}\|_1 \geq \|x\|_1 / \|Bx\|_1$ for every nonzero x , the exact quotient certifies a lower bound.

The largest residual bound is 6.0×10^{-3} , and refinement stops within two steps. The enclosures have relative width at most 6.0×10^{-3} for Base, 4.8×10^{-3} for Flat-L2 and 2.0×10^{-5} for Flat-GM, and the resulting bound on the Flat-GM/Base ratio agrees with the estimated ratio to 2.1×10^{-5} in all eight models (Table S2). Twenty-three of the 24 estimates lie inside their enclosures. The Flat-L2 estimate for Full instance046 is 2.4 times smaller than its certified lower bound, as a lower-bound estimator can be; this makes Flat-L2 less favorable, not more. Outputs are in `precision-check/` within the audit directory.

Archived fixed-incumbent computation. An earlier computation (`results-revision/fixed-basis/`) used a different reference basis: Gurobi solved each Base MIP (relative gap 10^{-3} , 16 threads), fixed the integers at its incumbent and took the basis of the resulting LP. Its archived medians point in the same direction with smaller magnitudes; for SA-Mat relative to Base they are 1.2×10^{-3} (Simple), 3.6×10^{-4} (Full) and 0.41 (SG-Ter-Mer). The magnitudes therefore depend on the reference basis. That computation did not retain its exports, basis identities or binary hash, and its multithreaded incumbents cannot be replayed, so it is not used as evidence. The provenance audit is in `results-revision/research-audit/fixed-basis-recovery-20260914-v2/`.

Table S2: Precision check on the stress sample. “Base $\kappa_1 \geq$ ” is the certified lower bound. “Enclosure width” is the largest relative width (upper over lower bound, minus one) among Base, Flat-GM and Flat-L2. The last two columns give the estimated ratio $\kappa_1(\text{Flat-GM})/\kappa_1(\text{Base})$ and its upper bound, the a posteriori upper bound for Flat-GM divided by the certified lower bound for Base.

Class	Model	Base $\kappa_1 \geq$	Enclosure width	Flat-GM/Base estimate	Upper bound
Simple	caso32e	9.3×10^{19}	1.5×10^{-4}	2.405×10^{-6}	2.405×10^{-6}
Simple	caso05f	1.0×10^{19}	3.7×10^{-3}	1.681×10^{-3}	1.681×10^{-3}
Simple	caso27f	8.7×10^{18}	1.9×10^{-5}	1.077×10^{-7}	1.077×10^{-7}
SG-Ter-Mer	instance067	1.2×10^{21}	4.1×10^{-3}	6.247×10^{-6}	6.247×10^{-6}
SG-Ter-Mer	instance028	4.0×10^{11}	1.5×10^{-6}	1.216×10^{-2}	1.216×10^{-2}
SG-Ter-Mer	instance074	4.0×10^{20}	9.5×10^{-5}	2.277×10^{-9}	2.277×10^{-9}
Full	instance014	1.2×10^{22}	6.0×10^{-3}	3.055×10^{-6}	3.055×10^{-6}
Full	instance046	4.7×10^{20}	2.7×10^{-4}	6.046×10^{-7}	6.046×10^{-7}

Table S3: Computing environment by campaign. “Not recorded” is stated rather than reconstructed; no quantity reported from those campaigns depends on the host.

Campaign	Optimizer	Thr.	Limit	Machine	Reported quantity
Fixed-basis audit	HiGHS 1.15.1 (LP only)	1	—	not recorded	κ_1
Matrix audit	none	—	—	not recorded	κ_2^+ , identity
Operational campaign	Gurobi 13.0.1	16	1,800 s	Xeon Gold 6138, Debian 12	PAR10 (s), work units
Replication, seed 7	Gurobi 13.0.1	16	1,800 s	ClusterUY, node not recorded	work units
Replication, CPLEX	CPLEX 22.1.2.0	16	1,800 s	ClusterUY, node not recorded	deterministic ticks
Stress experiment	HiGHS 1.15.1, Gurobi 13.0.2, CPLEX 22.1.2.0	1	10 s	not recorded	verified counts

S3 Computing environment

Table S3 states, per campaign, the optimizer and version, the thread count, the time limit and the machine. Only one row of it can affect a reported number. The mean PAR10 of the operational campaign is the single quantity in this work stated in units of time, and all 240 of its assigned runs carry a native log naming the same processor and operating system; two further logs in `operational/precision-smoke/` come from a laptop and belong to a separate smoke test, not to the campaign. Everything else is reported in solver-internal units that do not depend on the host — Gurobi work units for the seed replication, CPLEX deterministic ticks for the solver replication — or in counts and condition numbers, which do not depend on it either. The two replications ran on ClusterUY under SLURM with 16 cores per task, but their node is not recorded, and the machine of the fixed-basis audit, of the matrix audit and of the stress experiment was not recorded either. No number taken from any of them would change if it had been.

S4 Operational evidence and reconstruction

The final historical campaign assigned 15 instances in each of two dispatch families, four policies, and two relative MIP gaps. Thus there are 240 assigned Gurobi 13.0.1 runs (Table S3). All use seed 42, 16 threads and a 1,800-second limit. The four policies are Base, Flat-GM, Flat-L2 and Role-Hybrid. The assigned instances are

Simple: caso01a, caso03b, caso05d, caso10e, caso14a, caso20c, caso22d, caso24f, caso27a, caso29c, caso31e, caso33f, caso42b, caso44c, caso46e.

SG-Ter-Mer: instance001, instance005, instance014, instance022, instance027,
instance031, instance042, instance046, instance051, instance058, instance068,
instance072, instance077, instance081, instance092.

They are those of the historical campaign and are spread across the ordering of each class, but the rule that produced them was not recorded, so this is a description of the subsets rather than evidence of a pre-registered design. What can be checked is that they predate the analysis: the runs are reconstructed from logs retrieved from the cluster, and the frozen tables of the two replications reuse the same two lists. The analysis retains the assigned subsets and includes every non-completion in the primary runtime endpoint.

For assigned instance i in a cell, let t_i be the recorded runtime and define $T_i = t_i$ for completion and $T_i = 18,000$ otherwise. The primary summary is $15^{-1} \sum_i T_i$. This PAR10 endpoint measures operational completion under the specified stopping criteria. It is not an exact primal or optimality certificate. In particular, a solver-completed point with a concerning original residual is still counted as completed, with feasibility diagnostics reported separately.

Scenario defect in the SG-Ter-Mer instances. The market dispatch of this family reads a local demand series and a total demand series. In the historical instances the total series was rebuilt from a noisy rescaling of residual demand, and in 39 of the 68 instances it falls below the local demand in at least one hour: 9 of the 15 instances assigned above, among them instance077, where the inversion occurs in 22 hours and reaches 259 units. The exported models remain well-posed MILPs and every comparison in this paper is between representations of the same model, so no reported equivalence, residual or effort measurement is affected. What the defect does mean is that those instances are not faithful market scenarios and that the physical reading of their solutions is unreliable. A corrected version of the family, rebuilt from the authoritative demand series, exists outside this work and shows no inversion; no result reported here uses it. The counts above are produced by `scripts/research/check_sgtm_demand.py` from the instance files in the archive.

Does the defect drive the SG-Ter-Mer result? Six of the fifteen instances are free of the inversion. Table S4 repeats the two endpoints on that subset, with the same work measure and the same common-completion pool as the main text. The conditional gain does not depend on the defective scenarios: the Flat-GM work ratio is 0.452 at a 1% gap and 0.502 at 0.1% on the six, against 0.698 and 0.682 on all fifteen. The failure-sensitive reversal does depend on them, for a reason worth stating: the instance that times out under every scaled policy, instance077, is one of the nine, so on the defect-free subset every scaled policy completes and PAR10 follows the conditional gain instead of contradicting it. The reversal reported in the main text therefore rests on a single instance, which is what makes it a statement about failure sensitivity rather than about typical effort.

Work and uncertainty. The old runner’s scaled-policy Work markers inherited a fixed, zero-decimal stream format. Native Gurobi summaries instead retain two decimals. The reconstruction uses those native summaries for every policy, at their common 0.01-unit resolution. It never combines a Work value with a timeout penalty measured in seconds. Byte equality of rounded Work or node counts cannot prove equality of exported models.

Secondary Work ratios use only the instances completed by every policy in a given family/gap cell. Ratios are paired by instance; their geometric mean is $\exp(n^{-1} \sum_i \log(W_{i,p}/W_{i,\text{Base}}))$. Percentile intervals use 10,000 paired bootstrap samples, seed 20260908. These are exploratory intervals, without multiplicity adjustment, and are conditional on the common completion pool. A separate

Table S4: SG-Ter-Mer reconstruction on all fifteen assigned instances and on the six without the demand inversion. Work ratios are geometric means against Base on the instances every policy completed; PAR10 is the arithmetic mean over the assigned pool, with 18,000 seconds for a non-completion.

Gap	Policy	Work ratio vs Base		Mean PAR10 (s)	
		All 15	Defect-free 6	All 15	Defect-free 6
1%	Base	—	—	116.2	22.8
	Flat-GM	0.698	0.452	1216.5	5.9
	Flat-L2	0.863	0.844	1217.6	14.4
	Role-Hybrid	0.745	0.470	1217.3	5.4
0.1%	Base	—	—	134.1	25.0
	Flat-GM	0.682	0.502	1216.2	7.1
	Flat-L2	0.802	0.838	1216.0	13.6
	Role-Hybrid	0.726	0.493	1216.9	6.2

Table S5: Reconstructed operational campaign. Arithmetic mean PAR10 in seconds on the 15 assigned instances per cell; a timeout contributes 18,000 seconds. Parentheses give completed runs. All rows concern Gurobi.

Class	Gap	Base	Flat-GM	Flat-L2	Role-Hybrid
Simple	1%	11.90 (15)	2.12 (15)	20.44 (15)	2.11 (15)
Simple	0.1%	52.00 (15)	72.98 (15)	33.17 (15)	72.63 (15)
SG-Ter-Mer	1%	116.15 (15)	1216.51 (14)	1217.60 (14)	1217.34 (14)
SG-Ter-Mer	0.1%	134.08 (15)	1216.24 (14)	1216.05 (14)	1216.91 (14)

sensitivity calculation uses the intervals of width 0.01 implied by rounding each native Work value. Neither calculation accounts for the excluded timeout’s unobserved completion effort.

Flat-GM/Base on the 14 common completed SG-Ter-Mer instances has ratios 0.698 with interval $[0.486, 0.944]$ at 1% gap, and 0.682 with $[0.470, 0.925]$ at 0.1%. However, instance077 completes under Base and times out under all three scaled policies at both gaps. For Simple at 1%, Flat-L2/Base has Work ratio 1.792 with interval $[1.033, 3.542]$. These conditional observations do not establish a uniform solver advantage. Table S5 gives the complete PAR10 summary.

Replication under a second seed and a second solver. The reconstruction fixes the solver, the seed and the thread count. To separate what is stable from what belongs to that configuration, the same 30 instances, four policies, two gaps, 1,800-second limit and 16 threads were rerun twice, changing one factor at a time: Gurobi with seed 7, and CPLEX with the original seed 42. The run tables were frozen before the runs, the policy order within an instance is seeded by the instance digest, and the records carry the same verification as the campaign. Table S6 reports the paired effort ratio against Base on the instances every policy completed. Effort is each solver’s own measure, work units for Gurobi and deterministic ticks for CPLEX, so a column is read within its solver and never across. Runs classified as infrastructure failures were repeated and solver timeouts were not; there were two such episodes, the eight Gurobi runs of one instance that failed while the licence server was saturated, and the eight CPLEX runs of instance077, repeated with more memory after one of them was killed by the cgroup limit. The instance that reverses the PAR10 ordering in the reconstruction, instance077, behaves differently in every condition: under Gurobi at seed 7 all eight of its runs reach the limit, Base included, while under CPLEX the run that reaches the limit is

Table S6: Paired effort ratio against Base on the instances completed by every policy, with bootstrap intervals, for the reconstruction and its two replications. Values below one favour the scaled policy.

	Gurobi, seed 42	Gurobi, seed 7	CPLEX, seed 42
<i>Simple, gap 1%</i>			
Flat-GM	0.631 [0.31, 1.03]	0.783 [0.47, 1.03]	0.609 [0.28, 1.04]
Flat-L2	1.792 [1.03, 3.54]	2.226 [1.16, 4.70]	1.306 [0.39, 4.26]
Role-Hybrid	0.631 [0.31, 1.03]	0.783 [0.47, 1.04]	0.609 [0.28, 1.04]
<i>Simple, gap 0.1%</i>			
Flat-GM	1.063 [0.89, 1.25]	1.002 [0.91, 1.10]	0.837 [0.53, 1.15]
Flat-L2	1.385 [0.90, 2.33]	1.524 [1.05, 2.41]	1.425 [0.52, 3.64]
Role-Hybrid	1.063 [0.89, 1.24]	1.002 [0.91, 1.10]	0.837 [0.53, 1.14]
<i>SG-Ter-Mer, gap 1%</i>			
Flat-GM	0.698 [0.49, 0.94]	0.778 [0.53, 1.06]	1.078 [0.99, 1.18]
Flat-L2	0.863 [0.72, 1.03]	1.115 [0.92, 1.38]	0.955 [0.92, 0.99]
Role-Hybrid	0.744 [0.49, 1.04]	0.775 [0.53, 1.06]	1.053 [0.98, 1.15]
<i>SG-Ter-Mer, gap 0.1%</i>			
Flat-GM	0.682 [0.47, 0.92]	0.777 [0.53, 1.06]	1.077 [0.99, 1.18]
Flat-L2	0.802 [0.65, 0.98]	1.110 [0.93, 1.36]	0.994 [0.93, 1.09]
Role-Hybrid	0.726 [0.48, 1.00]	0.779 [0.53, 1.06]	1.052 [0.97, 1.16]

Base at the 1% gap and Role-Hybrid at 0.1%.

Original residual normalization. For an equality, the absolute residual is $|a_r^T x - b_r|$. The RHS-normalized residual divides by $1 + |b_r|$, whereas the activity-normalized residual divides by $\max\{1, |b_r|, \sum_j |a_{rj} x_j|\}$. For inequalities the positive violation replaces the absolute equality residual. Neither is a supplied engineering acceptance budget. For caso46e, Flat-L2 at 1% produces an absolute equality residual about 4.97 on a row with zero RHS. Its RHS-normalized value is still 4.97, while the run’s maximum activity-normalized residual is 1.15×10^{-7} . Calling the former a 497% physical error would be unjustified.

The directory `results-revision/research-audit/operational/` contains native cluster logs, the reconstructed run table, cell summaries, paired contrasts, original input hashes and analysis settings. The script `scripts/audit_operational_evidence.py` generates these summaries and the main-text table. A separate two-run smoke check under `precision-smoke/` verifies that the corrected C++ Work marker preserves the solver API’s precision; it does not replace any of the 240 historical observations.

S5 Fresh matrix audit

Ten seeds, 20260909–20260918, are crossed with exponent ranges $[-R, R]$ for $R \in \{1, 2, 3\}$, two disturbance severities (3 and 6), and three coupling patterns: `coupling`, `coupling_uniform`, and `mixed`. There are 180 instances and six exports per instance: Base, SA-Mat, Flat-GM, Flat-L2, Role-Hybrid and SA-Mat-permB. A solver-free DummyLp backend writes the 1,080 LP files.

The parser compares named rows and columns, coefficient supports, row senses, right-hand sides, and the remaining objective/domain sections. It infers each row’s positive factor from a nonzero pivot coefficient and checks the entire row and RHS at relative tolerance 10^{-13} . All final exports pass. This remains a floating-point proportionality check; it is not a rational proof of exact scaling.

Byte hashes supply the stronger literal-identity claims where they hold.

The numerical reference rank is determined from row-L2-normalized Base using a relative threshold 10^{-12} . Every variant then uses that same rank for the spectral diagnostic. Independent rank thresholds on each export disagree on 25 of 1,080 cases. Very large unscaled condition estimates near binary64 resolution are therefore not interpreted quantitatively. The paired scaled diagnostics are below 6.5×10^6 .

The authoritative directory is `results-revision/research-audit/identifiability-final/`. Its manifest records the generator binary and script hashes, settings, named comparisons and all exports. The retained `identifiability-confirmatory/` directory used the former fixed precision serializer and is superseded for current matrix claims.

S6 Frozen binary stress protocol

The experiment crosses three binary families, eight seeds (100–107), and four coefficient exponents (0, 3, 6, 9): 96 models with 12 binary variables each. Each model is run under two presolve settings, three solvers, and nine policies, giving 5,184 planned configurations. Budget-round abstains on 432 configurations before calling an optimizer; hence there are 4,752 actual calls. These configurations are not statistically independent observations.

For seed s , NumPy’s default generator is seeded by $20260908 + s$. The cardinality family has one all-ones row. Packing uses three rows of uniformly sampled integers 1 through 9. Signed rows use integers from -9 through 9, replacing zeros by one. For row r , $k_r = \max\{2, \lfloor 0.55 \sum_j \max(W_{rj}, 0) \rfloor\}$; profits are uniform integers 1 through 30. The represented constraints are $10^e Wx \leq 10^e k - 0.11$. Integer enumeration proves the binary optimum for each stored model. Changing e also changes the relaxation; it is not pure row re-emission. The budget is 0.01 in the declared canonical stress-test units.

Both solvers use one thread, the instance seed, a ten-second limit and zero target absolute and relative MIP gaps. Primal and integrality tolerances are 10^{-6} , except GM-tight, which uses 10^{-9} . Gurobi’s enabled-presolve arm uses its automatic presolve setting; HiGHS uses presolve on. Internal solver scaling remains at its default. Recorded versions are Python 3.10.12, NumPy 2.2.6, HiGHS 1.15.1 and Gurobi 13.0.2. Solving uses the Python APIs rather than text-file import.

Base, GM and L2 scale all rows according to their stated kernels, without a skip band. Budget-only uses ϵ/δ_r . Budget-GM projects GM onto the joint budget/coefficient interval with window $[10^{-9}, 10^9]$. Budget-round additionally reserves $10^{-6} \sum_j |a_{rj}|$ for rounding and rejects the whole model if any row’s contract is incompatible. Dyadic-GM imposes the coefficient window and exact binary64 multiplication, including the RHS; Budget-dyadic also imposes the residual budget. The latter two policies were added after the initial floating-endpoint audit and are exploratory, not a new holdout experiment.

The sequence of preserved datasets under `results-revision/research-audit/` is:

Directory	Interpretation
<code>budget-pilot</code>	Single exploratory seed 0; six initial policies.
<code>budget-confirmatory</code>	Seeds 100–107; seven policies; original nearest-rounded endpoint.
<code>budget-confirmatory-v2</code>	Same seven policies and seeds; exact endpoints rounded inward.
<code>budget-final</code>	Authoritative nine-policy run; inward endpoints and two exploratory dyadic controls.

An attractive result from an intermediate implementation is not substituted for the corrected final result. In particular, the earlier HiGHS Budget-GM count of 191 verified configurations becomes 182 with correctly inward-rounded endpoints.

Table S7: Enumerated stress experiment, 192 planned configurations per policy and solver. “Verified” means that the returned point passes the declared acceptance rule and its rounded assignment attains the independently enumerated optimum. “Other” includes unverified outputs and solver errors; “Abstain” means no optimizer call was attempted. Abstention is not a performance success. The separated GCD-floor row is the later integer-aware control, with the same denominator and acceptance rule.

Method	HiGHS 1.15.1			Gurobi 13.0.2			CPLEX 22.1.2		
	Verified	Other	Abstain	Verified	Other	Abstain	Verified	Other	Abstain
Base	167	25	0	156	36	0	150	42	0
GM	141	51	0	140	52	0	140	52	0
L2	141	51	0	139	53	0	140	52	0
GM-tight	166	26	0	166	26	0	166	26	0
Dyadic-GM	142	50	0	140	52	0	140	52	0
Budget-only	191	1	0	151	41	0	140	52	0
Budget-GM	182	10	0	152	40	0	140	52	0
Budget-dyadic	161	31	0	153	39	0	140	52	0
Budget-round	48	0	144	48	0	144	48	0	144
GCD-floor	192	0	0	192	0	0	192	0	0

Paired outcomes and factor distances. The paired comparison in the main text uses identical model, presolve and solver keys; its counts are regenerated by `scripts/research/analyze_budget_experiment.py`, which also writes `budget-final/paired-discordance.json`. Budget-only and Budget-GM select different factors at multipliers 10^0 and 10^3 , where both verify every configuration on all three solvers. At 10^6 and 10^9 the budget endpoint binds, and Budget-GM returns the inward-rounded endpoint, at most one unit in the last place from Budget-only’s factor ϵ/δ on every row of every model. All ten configurations whose verification outcomes differ between the two policies lie at these multipliers. The failure categories quoted in the main text are taken from `results-revision/research-audit/exploration-failures/summary.json` for HiGHS and Gurobi and `exploration-failures-cplex/summary.json` for CPLEX, both written by the same script, which classifies each returned point with an infeasible rounded assignment by whether it violates the original row budget and whether it violates the exported tolerance.

Classical integer-row controls. The follow-up in `results-revision/research-audit/exploration-lattice/` uses the same 96 stored models, three solvers and two presolve settings. For an integral row $a^T z \leq b$, let $g = \gcd_j |a_j|$ and replace it by $(a/g)^T z \leq \lfloor b/g \rfloor$. Exact rational division of the stored RHS determines the floor. GCD-floor exports this row directly; GCD-floor-GM adds floating GM normalization; GCD-midgap-GM first replaces the floored RHS by $\lfloor b/g \rfloor + 1/2$ and then applies GM. The three controls account for 1,728 further calls, all yielding verified original-model optima. The midpoint operation is integer-equivalent before multiplication and weakens the LP relative to flooring. These are established reductions or arithmetic controls, not new scaling proposals.

The direct GCD-floor export preserves all 96 binary feasible sets exactly and verifies 192/192 configurations per solver. Independent enumeration of the actual binary64 exports also finds equivalence for all 96 GCD-midgap-GM models, but detects excluded feasible boundary points in 60 of the GCD-floor-GM models. Thus successful returned optima do not establish equivalence of every represented constraint. The main comparison foregrounds GCD-floor, whose transformation and export are both exact for these inputs. This arithmetic distinction also explains why an exact-export

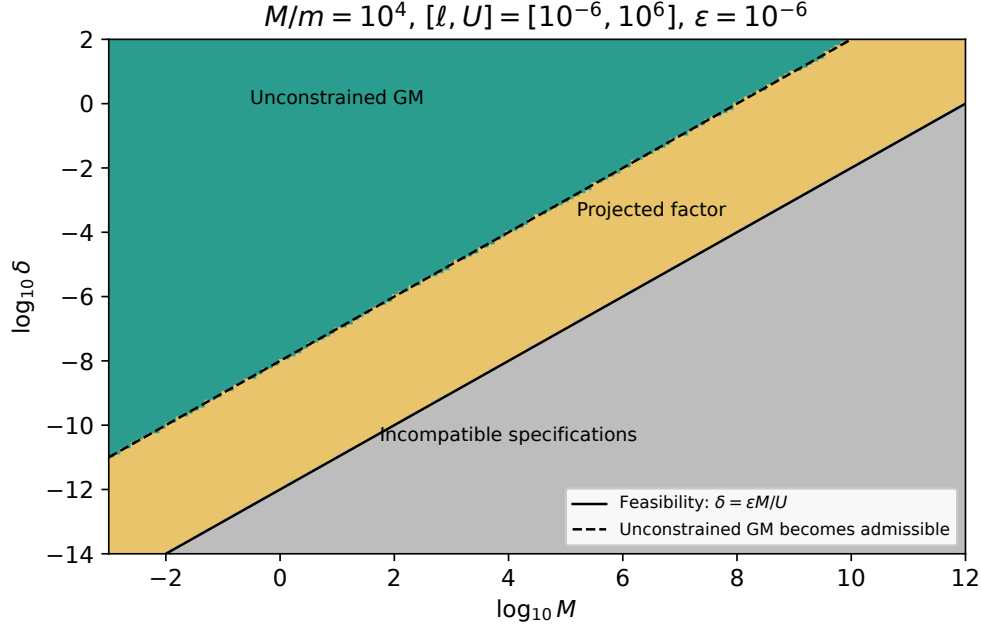


Figure S1: Exact compatibility regions for a row with fixed within-row spread. Below the solid line, the residual budget conflicts with the coefficient ceiling. In the middle region the constrained minimax factor is projected onto the admissible interval; above the dashed line ordinary GM already satisfies it. This diagram concerns the conditional row contract, before reserving any integer-rounding allowance.

condition is separate from selecting a mathematically admissible real factor.

S7 The declared budgets of the application table

The budgets of the application table in the main text are declared by a modeller, not measured from an operating tolerance, so the natural objection is that admissibility was read off a convenient choice. Table S7 scales all six declared budgets by a common multiplier and reports, per constraint group, the tightest multiplier at which each kernel is still admissible on every row of the group. Admissibility is monotone in the budget — a larger δ only widens the interval of Proposition 5.1 — so the tightest multiplier with full coverage is the point at which the contract starts to bind, and the data confirm the monotonicity on every group and kernel.

All three kernels remain admissible on all 4,320 rows down to a multiplier of 10^{-4} : the declared budgets could be tightened ten thousandfold before any of them loses a row. The two kernels the main text compares, the unit factor of Base and the geometric mean of Flat-GM, survive to 10^{-5} . Below that the groups separate in the way the proposition predicts: the water balances, whose coefficients span the widest range, keep admitting the unit factor down to 10^{-9} , while the Euclidean kernel of Flat-L2 is the first to leave the admissible interval on the four groups written in megawatts. The conclusion of the main text is therefore not an artifact of the particular numbers declared; it would take a tolerance four orders of magnitude tighter than a tenth of a megawatt to make the choice of kernel a question of admissibility rather than of conditioning.

The analysis is reproducible without an optimizer. The exported LP it reads is stored beside the results, `results-revision/research-audit/application-budgets/caso46e-dummylp.lp.gz`,

Table S8: Tightening the declared budgets of the application table. For each constraint group of caso46e, the smallest common multiplier of the declared budgets at which the kernel is still admissible on every row of the group, with $\epsilon = 10^{-6}$ and the coefficient window $[10^{-9}, 10^9]$. A multiplier of one is the budget declared in the main text; smaller means a tighter tolerance. The sweep covers 10^{-16} to 10^2 .

Constraint group	Rows	Tightest admissible multiplier		
		Base	Flat-GM	Flat-L2
Generation identities, demand cap	1,080	10^{-5}	10^{-5}	10^{-4}
Generation definition, Bonete/Palmar	720	10^{-5}	10^{-5}	10^{-4}
Production envelope tangents	1,080	10^{-5}	10^{-5}	10^{-4}
Water balance, Bonete/Palmar	720	10^{-9}	10^{-7}	10^{-5}
Water balance, Baygorria	360	10^{-9}	10^{-5}	10^{-5}
Generation definition, Baygorria	360	10^{-12}	10^{-9}	10^{-4}
All groups	4,320	10^{-5}	10^{-5}	10^{-4}

Table S9: Cross-run certificate audit by level and policy. “Dual bound” counts runs whose reported bound lies above a verified point of the same model; “Certificate” counts the subset that also reported completion further from that point than the requested gap. Runs and instances are the totals of that level.

Campaign	Level	Runs	Inst.	Base	Flat-GM	Flat-L2	Role-Hybrid
Stopped campaign	Dual bound	77	27	6	28	13	30
	Certificate	25	9	5	9	1	10
Operational replication	Dual bound	20	6	1	6	3	10
	Certificate	9	4	1	3	1	4

and `make research-application-export` re-exports it from the instance with the solver-free build and compares the two byte by byte; `make research-application-budgets` regenerates both tables from it. That export is written by the repository’s own serializer rather than by Gurobi, as the archived one was: the two differ in the last digits of their decimal output, which moves the reported quantities of one group by a relative 10^{-15} and nothing in either table.

S8 Cross-run certificate audit

The audit of §7.5 of the article uses two stored campaigns: the solver-robustness campaign, which was prepared, partly run and then stopped by decision, and the two replications of the operational reconstruction. Neither is a census and neither was designed to estimate how often a solver contradicts itself, so the counts below describe what occurred in the runs that exist. Table S9 splits them by level and policy.

The comparison rests on the stored verification. Each run’s returned point is re-evaluated against the original unscaled model, so a point accepted on a scaled export is still checked where it has to hold, and `verif_solver_obj_original` is its objective there. Points accepted at the campaign tolerance can be slightly infeasible, so the audit also carries a strict tier that only trusts points whose residuals are at round-off; the counts reported in the article use the ordinary tier, and the extreme case below is contradicted under both.

The extreme case. On `v1_simple_000293`, one of the recombined Simple models, eight runs exist at a 1% gap and seed 1: four policies under each of Gurobi 13.0.1 and CPLEX 22.1.2, all recorded against the same `instance_sha256`. Seven of them return an objective between -2.893×10^8 and -2.913×10^8 . The eighth, Gurobi on the unscaled Base export, returns -1.317×10^8 with status optimal, a reported gap of zero and a dual bound equal to its objective. Its stored record shows one node explored against a root gap of 1.22, a wall time of 6 s, and three solver warnings: large bounds, large matrix coefficients and large right-hand sides. Its returned point is feasible for the original model with maximum relative violation 1.2×10^{-7} and relative activity violation 3.2×10^{-14} , so the point is sound and the bound is not. CPLEX on the same unscaled file explored 6,151 nodes, emitted no warnings and reached -2.904×10^8 . The best verified point of the instance, -2.913×10^8 , comes from Gurobi on the Flat-L2 export.

Two readings are excluded by the record. The runs are not on different models, because the instance hash is identical across all eight. And the discrepancy is not an artifact of scaling, because the run that fails is the unscaled one and the same solver on two scaled exports of the same model does not fail. What the record does not establish is a cause inside the solver, which is not observable from these logs; the warnings and the single node are consistent with a tolerance-driven termination on a badly scaled model, and that is as far as the evidence goes.

S9 Independent verification and arithmetic checks

The independent script `scripts/research/verify_saved_experiment.py` imports neither the experiment runner nor an optimizer. For each of the 96 stored models it enumerates all 4,096 binary assignments using integer activities, checks the reported optimum, and then recomputes the acceptance decision from the saved point. It checks all 3,456 distinct Cartesian-product configuration keys, all 288 abstentions, factor admissibility, and the 768 dyadic exports with exact rational multiplication.

A verified optimum requires an exactly feasible rounded binary point attaining the enumerated objective, a returned point within 10^{-9} of that rounded point, and bounds respected within 10^{-9} . Raw-point residual budgets and integrality errors are separate diagnostics. In particular, the verified-optimum label concerns the rounded assignment; it does not claim exact feasibility of the raw floating-point point. A failure of this conservative check does not by itself identify a solver bug.

Rational checks interpret each stored binary64 datum exactly. They do not recover unrecorded real-valued physical data. Arbitrary-factor multiplication may change the represented row by rounding even when its factor lies in the exact admissible interval. The dyadic routine avoids this by intersecting the coefficient/budget interval with exact exponent restrictions on every coefficient and the RHS. It compares candidate centering objectives as rational numbers, because floating logarithms can misclassify a near-midpoint as a tie.

Tests compare continuous interval projection with an independent linear program in log coordinates, check positive re-emission, incompatible specifications, subnormal and overflow boundaries, and a cancellation example where a floating-point residual misses an exact unit violation. A native C++ integration test exercises the actual LP writer with very small coefficients, RHS, bounds and objective values. The serializer uses `max_digits10` and general decimal notation so binary64 values survive decimal round-trip parsing. Serialization correctness and exact multiplication are separate conditions; solver-internal transformations remain outside both guarantees.

S10 Reproduction and artifact layout

The root README and `environment/research-requirements.txt` specify the recorded analysis environment. The following commands need no optimizer license:

```
make research-check-python # tests, saved binary audit, matrix hashes
make research-check # also builds/tests the native C++ exporter
make research-analysis # current tables and vector figures
make research-fixed-basis-check # rebuild all 1,530 bases from LP text
make paper # article and this supplement
```

The reproducibility archive carries the analyses, the aggregates and the inputs they read, and lists in `EXCLUDED.txt` the bulk files it leaves out: the exported LP text and bases of the fixed-basis audit, about 5 GB that stay in the repository. The fixed-basis rebuild reads those exports and therefore needs them. The full export check needs GNU Make and a C++17 compiler. PDF compilation uses pdfLaTeX, BibTeX and standard TeX packages, without the old proprietary-font requirements. Figures are generated with matplotlib from stored data or explicit mathematical expressions.

Fresh experiments write to separate directories and are not required for verification:

```
make research-grid GRID_OUT=results-revision/research-audit/reruns/grid
make research-fixed-basis \
  FIXED_BASIS_OUT=results-revision/research-audit/reruns/fixed-basis
make research-budget BUDGET_SOLVERS=highs \
  BUDGET_OUT=results-revision/research-audit/reruns/highs
```

The default two-solver budget rerun also requires Gurobi and a suitable license. Runtime measurements need not reproduce bit for bit across hardware or solver versions. Source snapshots and hashes in the final budget directory identify the implementation that generated the reported results; later arithmetic edge-case repairs are recorded separately, with a comparison against all saved row factors. The accompanying source/data archive (Online Resource 2) preserves the inputs needed to regenerate the current main tables and figures.